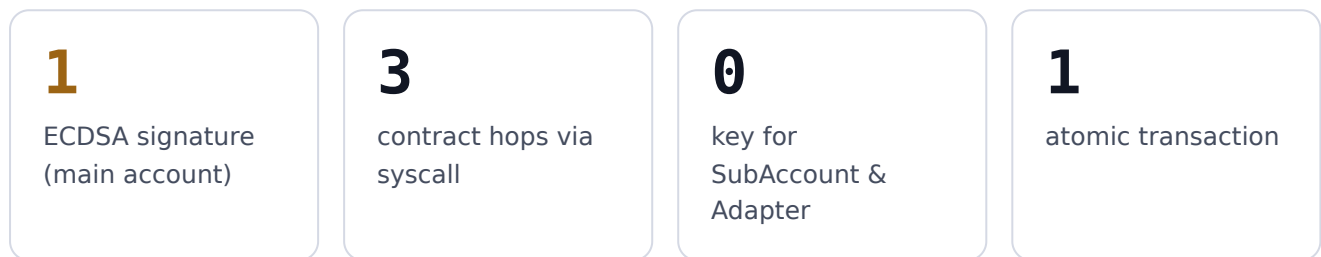
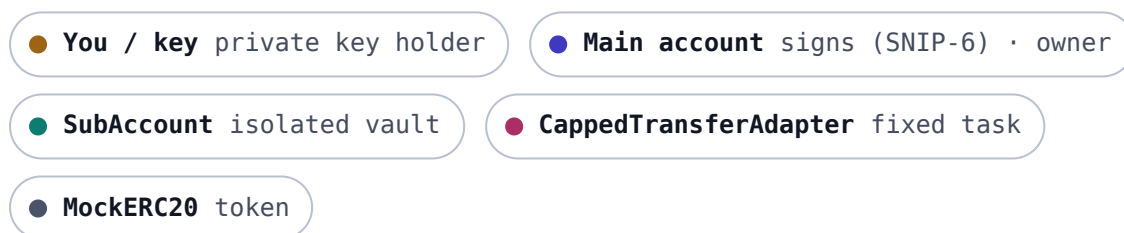


Who signs, and when?

A **single signature** — the main account's — propagates through several contracts inside **one atomic transaction**. The sub-account and the adapter hold **no private key**: they authorize by reading `get_caller_address()`, not a signature.



The actors 5 CONTRACTS / ROLES



Prerequisite — the setup ONCE

Four separate transactions, **each signed on its own by the main account** (through its `__execute__`). They arm the sub-account before the flow we care about.

1	<code>deploy MockERC20, SubAccount(owner), CappedTransferAdapter(token, recipient, cap)</code>	main account
2	<code>MockERC20.mint(subAccount, 1000)</code>	main account
3	<code>SubAccount.set_adapter(adapter, true)</code>	main account
4	<code>SubAccount.approve_token(token, adapter, 500)</code>	main account

Afterwards: the sub-account holds 1000 tokens, knows the adapter (whitelist), and has granted it a 500 allowance. The main account itself holds nothing at risk.

The execution ONE TRANSACTION

You want to move 200 tokens out of the sub-account through the adapter.
Here is the full chain, from the off-chain gesture to the actual movement of funds.



You sign a transaction with the main account's private key.

Payload: a single call → `SubAccount.execute(call)`.
This is the only signature in the whole chain.

SIGNATURE ×1 · OFF-CHAIN

▼ TRANSACTION SUBMITTED TO THE SEQUENCER ▼

1 ATOMIC TRANSACTION · ON-CHAIN

1 ● **Main account** ↻ `__validate__`

CHECKS the ECDSA signature matches the account's public key.

RESULT ✓ **valid signature**

2 ● **Main account** . `__execute__` → `call_contract_syscall`

● **SubAccount.execute(call)**

CALLER SEEN = **Main account**

GUARD `assert(get_caller_address() == owner)`

RESULT ✓ **caller = owner**

3 ● **SubAccount** ↻ `checks the whitelist`

GUARD `assert(adapters[call.to] == true)`

RESULT ✓ **target = authorized adapter**

4 ● **SubAccount** → `call_contract_syscall`

● **Adapter.pull_and_transfer(200)**

CALLER SEEN = **SubAccount** (the caller just changed)

5 ● **Adapter** ↺ checks the cap

GUARD

assert(200 <= cap)

RESULT

✓ amount ≤ cap

6 ● **Adapter** → transferFrom(subAccount, recipient, 200)

● **MockERC20**

CALLER SEEN

= **Adapter**

AUTHORIZED BY

the SubAccount → Adapter allowance set in setup step 4.

7 ● **MockERC20** ↺ updates balances

EFFECT

SubAccount **-200** · recipient **+200** · allowance **-200**.

RETURN

the value bubbles back up: ERC20 → Adapter → SubAccount → Main account.

The heart of the model: get_caller_address() shifts at every hop

None of these contracts signs. Each one decides simply by looking at who is calling it:

Main account → SubAccount → Adapter → ● ERC20 sees "Adapter"



Atomicity = security. Steps 1→7 live inside a single transaction. If any one assert fails (wrong owner, non-whitelisted adapter, cap exceeded), the whole transaction reverts: **no token moves**. The risk stays bounded to the allowance granted to the adapter, on the sub-account's balance alone.